



Universidad Interamericana De Panamá
Facultad De Ingeniería, Arquitectura Y Diseño

Materia:
Base de Datos II

Nombre / Cédula:
Ariel Torralbo - 8-1020-1890

Asignación:
Proyecto Final

Profesor:
Maryon Jose Torres Rodriguez

Tabla de Contenido

1. Introducción.....	4
1.1 Alcance del proyecto.....	4
1.2 Tecnologías utilizadas.....	4
2. Diagrama Entidad-Relación (DER)	5
2.1 Descripción de las entidades	5
2.2 Justificación de la Tercera Forma Normal (3FN).....	5
3. Caso de Uso y Explicación de Requisitos	7
3.1 Descripción del caso de uso	7
3.2 Requisitos funcionales y su implementación.....	7
3.3 Reglas de negocio clave	8
4. Documentación de las Herramientas Utilizadas	9
4.1 Base de Datos — MariaDB.....	9
4.1.1 Tablas.....	9
4.1.2 Vistas	10
4.1.3 Procedimientos almacenados	11
4.1.4 Datos de prueba.....	12
4.2 API REST — Node.js + Express + Sequelize	13
4.2.1 Dependencias principales.....	13
4.2.2 Endpoints disponibles	13
Especialidades — /api/especialidades	13
Médicos — /api/medicos	14
Pacientes — /api/pacientes.....	14
Citas — /api/citas.....	14
4.2.3 Ejemplo: agendar una cita usando el procedimiento almacenado.....	15
4.3 ETL — Carga de Pacientes desde Sistema Externo	16
4.3.1 Flujo del proceso.....	16
4.3.2 Extract.....	16
4.3.3 Transform.....	16
4.3.4 Load	17
4.3.5 Evidencia de ejecución real	17
4.3.6 Automatización	17
4.4 Visualización — Power BI.....	18
4.4.1 Conexión de Power BI a MariaDB.....	18
4.4.2 Contenido del dashboard.....	19

4.5 Pruebas — Funcionales y de Rendimiento.....	20
4.5.1 Pruebas funcionales (REST Client / Postman)	20
4.5.2 Pruebas de rendimiento (Apache JMeter).....	21
Configuración del Thread Group	21
Resultados obtenidos	21
5. Manual de Instalación y Ejecución — Paso a Paso	24
Paso 1 — Instalar y cargar la base de datos MariaDB.....	24
Paso 2 — Levantar la API REST	24
Paso 3 — Probar la API.....	25
Paso 4 — Configurar y ejecutar el ETL	25
Paso 5 — Construir el dashboard en Power BI	25
Paso 6 — Pruebas de carga con Apache JMeter.....	25
Resumen del orden de ejecución.....	26
6. Lecciones Aprendidas	27
6.1 PowerShell no interpreta la redirección '<' como bash	27
6.2 El ejecutable de MariaDB no estaba en el PATH del sistema	27
6.3 pip tampoco estaba disponible directamente en PATH	27
6.4 Rutas de proyecto de ejemplo no coinciden con la estructura real	27
6.5 La validación de choque de horario debía vivir en la base de datos, no solo en la API.....	28
6.6 Un ETL de calidad necesita reportar, no solo descartar, los datos inválidos	28
7. Glosario de Términos.....	29

1. Introducción

Este documento presenta la documentación técnica completa del Sistema de Gestión de Citas Médicas, desarrollado como proyecto final para el curso de Base de Datos II. El sistema fue diseñado para una clínica con múltiples especialidades, y cubre de forma integral el ciclo de vida de una solución de datos: desde el modelado relacional en tercera forma normal (3FN), pasando por una API REST para el acceso y manipulación de la información, un proceso ETL para la carga de datos externos, un tablero de visualización en Power BI, y un plan de pruebas funcionales y de rendimiento.

El objetivo de este documento es explicar qué se construyó, por qué se construyó de esa manera, y cómo se puede instalar, ejecutar y probar cada componente. Está organizado en las siguientes secciones: diagrama entidad-relación, caso de uso y requisitos, documentación de cada herramienta utilizada, manual de instalación paso a paso, lecciones aprendidas durante el desarrollo, y un glosario de términos técnicos.

1.1 Alcance del proyecto

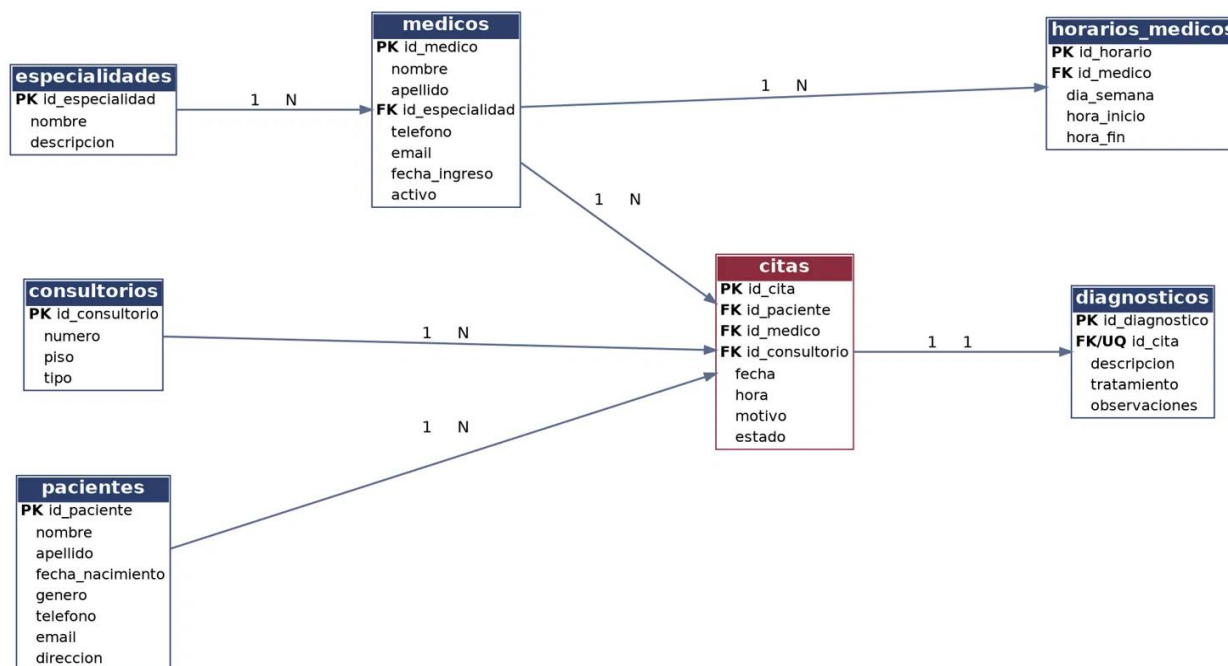
Fase	Entregable	Estado
1. Base de datos	Esquema MariaDB en 3FN + 2 vistas + 2 procedimientos almacenados	Completado
2. API REST	Node.js + Express + Sequelize, CRUD completo + consultas crudas	Completado
3. ETL	Script Python de carga y limpieza de pacientes desde CSV externo	Completado
4. Visualización	Conexión Power BI a MariaDB + dashboard	Completado
5. Pruebas	Colección Postman + plan de pruebas JMeter	Completado
6. Documentación	DER, manual, glosario, lecciones aprendidas	Este documento

1.2 Tecnologías utilizadas

Componente	Tecnología
Motor de base de datos	MariaDB 12.3
Backend / API	Node.js, Express, Sequelize, mysql2
ETL	Python 3.12, mysql-connector-python, python-dotenv
Visualización	Power BI Desktop
Pruebas funcionales	Postman / REST Client (VS Code)
Pruebas de rendimiento	Apache JMeter 5.6.3

2. Diagrama Entidad-Relación (DER)

El siguiente diagrama representa el modelo relacional completo de la base de datos clinica_citas. El modelo está compuesto por siete entidades: especialidades, medicos, pacientes, consultorios, horarios_medicos, citas y diagnosticos. Las relaciones se establecen mediante llaves foráneas (FK), y la cardinalidad se indica en cada línea de relación (1 a N, o 1 a 1 en el caso de citas–diagnósticos).



2.1 Descripción de las entidades

Entidad	Descripción	Relación principal
especialidades	Catálogo de especialidades médicas (Medicina General, Pediatría, Cardiología, etc.)	1 especialidad → N médicos
medicos	Personal médico de la clínica, cada uno vinculado a una única especialidad	1 médico → N horarios; 1 médico → N citas
consultorios	Espacios físicos donde se atienden las citas	1 consultorio → N citas
horarios_medicos	Franjas de disponibilidad semanal de cada médico (día + hora inicio/fin)	N horarios pertenecen a 1 médico
pacientes	Personas registradas que solicitan citas	1 paciente → N citas
citas	Tabla transaccional central: una paciente, médico, consultorio, fecha y hora	Entidad puente principal del sistema
diagnosticos	Resultado clínico de una cita ya atendida (descripción, tratamiento, observaciones)	1 cita → 1 diagnóstico (relación 1 a 1)

2.2 Justificación de la Tercera Forma Normal (3FN)

El modelo cumple con 3FN porque satisface las tres condiciones de normalización de forma progresiva:

- **1FN (valores atómicos):** cada columna almacena un único valor indivisible; no existen listas ni campos multivaluados (por ejemplo, el nombre completo del paciente se separa en nombre y apellido, en vez de un solo campo mixto).
- **2FN (dependencia total de la llave primaria):** todas las tablas usan una llave primaria simple (auto-incremental), por lo que no hay dependencias parciales posibles; cada atributo depende de la llave completa.
- **3FN (sin dependencias transitivas):** los datos que podrían repetirse se extrajeron a sus propias tablas. El caso más claro es especialidades: en vez de guardar el texto "Cardiología" en cada fila de medicos, se guarda solo el id_especialidad (FK), y el nombre real vive una sola vez en la tabla especialidades. Lo mismo aplica para consultorios frente a citas.

3. Caso de Uso y Explicación de Requisitos

3.1 Descripción del caso de uso

El sistema modela la operación de una clínica con múltiples especialidades médicas. La clínica necesita administrar pacientes, médicos, consultorios y las citas que se agendan entre ellos, además de registrar el diagnóstico resultante de cada consulta atendida. Este escenario se eligió porque presenta suficiente complejidad relacional (siete entidades interconectadas, relaciones 1 a N y 1 a 1, y una regla de negocio de exclusión mutua sobre horarios) para justificar tanto el diseño en 3FN como el uso de procedimientos almacenados y vistas.

Los actores principales del sistema son: el paciente (quien solicita y recibe citas), el médico (quien atiende las citas dentro de su especialidad y horario disponible), y el personal administrativo de la clínica (quien gestiona el catálogo de especialidades, consultorios y el ciclo de vida completo de las citas a través de la API).

3.2 Requisitos funcionales y su implementación

A continuación se detalla cada requisito funcional definido para el proyecto, y el componente técnico específico que lo resuelve.

Requisito funcional	Cómo se implementó
Registrar y consultar el catálogo de especialidades, médicos, pacientes y consultorios	CRUD completo (GET/POST/PUT/DELETE) para cada entidad, implementado con modelos Sequelize en la API REST (endpoints /api/especialidades, /api/medicos, /api/pacientes, /api/consultorios).
Agendar una cita sin permitir que un médico tenga dos citas en el mismo horario	Procedimiento almacenado sp_agendar_cita: antes de insertar, cuenta si ya existe una cita para ese id_medico, fecha y hora con estado distinto de 'Cancelada'; si existe, devuelve un mensaje de error y no inserta. Expuesto en la API como POST /api/citas/agendar, que responde HTTP 409 en caso de choque.
Evitar duplicar el nombre de la especialidad en cada médico (normalización)	Relación FK medicos.id_especialidad → especialidades.id_especialidad, en vez de guardar el texto de la especialidad directamente en la tabla medicos.
Consultar el detalle completo de una cita (paciente, médico, especialidad, consultorio) en una sola consulta	Vista vista_citas_completas, que hace JOIN de citas con pacientes, medicos, especialidades y consultorios. Expuesta en la API como GET /api/citas/completas, con filtros opcionales por fecha y estado.
Consultar el historial médico completo de un paciente (citas + diagnósticos)	Procedimiento almacenado sp_historial_paciente, que hace JOIN de citas, medicos, especialidades y diagnosticos (LEFT JOIN, porque no toda cita tiene diagnóstico aún) para un id_paciente dado. Expuesto como GET /api/pacientes/:id/historial.
Obtener estadísticas de desempeño por médico (total de citas, atendidas, canceladas, inasistencias)	Vista vista_estadisticas_medico, que agrupa citas por médico usando COUNT y SUM condicional (CASE WHEN). Expuesta como GET /api/medicos/estadisticas y consumida directamente por el dashboard de Power BI.

Requisito funcional	Cómo se implementó
Registrar el diagnóstico de una cita ya atendida, garantizando que cada cita tenga como máximo un diagnóstico	Restricción UNIQUE sobre diagnosticos.id_cita (relación 1 a 1 real a nivel de base de datos, no solo de aplicación), junto con endpoint POST /api/diagnosticos.
Registrar el horario de disponibilidad semanal de cada médico	Tabla horarios_medicos, con FK a medicos y una restricción CHECK (chk_horario) que obliga a que hora_fin sea mayor que hora_inicio.
Cargar pacientes provenientes de un sistema externo, limpiando y validando los datos antes de insertarlos	Script ETL etl_pacientes.py: extrae un CSV externo, transforma (normaliza fechas, teléfonos, género, valida correos, descarta duplicados) y carga a la tabla pacientes con INSERT ... ON DUPLICATE KEY UPDATE para que el proceso sea idempotente.
Visualizar de forma gráfica la operación de la clínica (ocupación por médico, distribución de citas, etc.)	Dashboard en Power BI conectado directamente a MariaDB, construido sobre las vistas vista_citas_completas y vista_estadisticas_medico.
Garantizar que la API responde correctamente y soporta carga concurrente	Pruebas funcionales con Postman / REST Client sobre todos los endpoints (incluyendo casos de error esperados, como el choque de horario), y pruebas de carga con JMeter sobre los 6 endpoints principales.

3.3 Reglas de negocio clave

- Un médico no puede tener dos citas activas en la misma fecha y hora (validado a nivel de aplicación por sp_agendar_cita, y reforzado a nivel de base de datos con la restricción UNIQUE uq_medico_fecha_hora sobre id_medico, fecha, hora).
- Un médico pertenece a exactamente una especialidad; una especialidad puede tener varios médicos.
- Una cita solo puede tener un diagnóstico, y un diagnóstico pertenece a una única cita (relación 1 a 1, forzada con UNIQUE en id_cita).
- El estado de una cita sigue un ciclo controlado por un ENUM: Programada, Confirmada, Atendida, Cancelada, No asistió — evitando valores libres o inconsistentes.
- No se permite eliminar un médico, paciente o consultorio si tiene citas asociadas (ON DELETE RESTRICT), para preservar la integridad del historial clínico. En cambio, los horarios de un médico sí se eliminan en cascada si el médico se elimina (ON DELETE CASCADE), ya que no tienen valor por sí solos.

4. Documentación de las Herramientas Utilizadas

Esta sección documenta en detalle cada componente técnico del proyecto: qué hace, cómo está construido, y los fragmentos de código más relevantes.

4.1 Base de Datos — MariaDB

El esquema completo vive en el archivo 01_schema.sql y crea la base de datos clinica_citas desde cero (DROP DATABASE IF EXISTS + CREATE DATABASE), con codificación utf8mb4 para soportar acentos y caracteres especiales del español.

4.1.1 Tablas

Las siete tablas del modelo, todas bajo el motor InnoDB (soporta llaves foráneas y transacciones):

```
CREATE TABLE especialidades (
  id_especialidad INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL UNIQUE,
  descripcion VARCHAR(255)
) ENGINE=InnoDB;

-- Consultorios físicos de la clínica
CREATE TABLE consultorios (
  id_consultorio INT AUTO_INCREMENT PRIMARY KEY,
  numero VARCHAR(10) NOT NULL UNIQUE,
  piso TINYINT NOT NULL,
  tipo VARCHAR(50) DEFAULT 'General'
) ENGINE=InnoDB;

-- Médicos (cada médico tiene UNA especialidad -> FK, no se repite texto -> 3FN)
CREATE TABLE medicos (
  id_medico INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  apellido VARCHAR(60) NOT NULL,
  id_especialidad INT NOT NULL,
  telefono VARCHAR(20),
  email VARCHAR(100) UNIQUE,
  fecha_ingreso DATE NOT NULL DEFAULT (CURRENT_DATE),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_medico_especialidad FOREIGN KEY (id_especialidad)
    REFERENCES especialidades(id_especialidad)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Las demás tablas las puede visualizar en el link de github al final del documento

4.1.2 Vistas

Vista 1: vista_citas_completas: Junta cita, paciente, médico, especialidad y consultorio en una sola fila. Es la vista que consume tanto la API (endpoint /api/citas/completas) como el dashboard de Power BI.

```
CREATE OR REPLACE VIEW vista_citas_completas AS
SELECT
    c.id_cita,
    c.fecha,
    c.hora,
    c.estado,
    c.motivo,
    p.id_paciente,
    CONCAT(p.nombre, ' ', p.apellido) AS paciente,
    p.telefono AS telefono_paciente,
    m.id_medico,
    CONCAT(m.nombre, ' ', m.apellido) AS medico,
    e.nombre AS especialidad,
    co.numero AS consultorio,
    co.piso
FROM citas c
JOIN pacientes p ON p.id_paciente = c.id_paciente
JOIN medicos m ON m.id_medico = c.id_medico
JOIN especialidades e ON e.id_especialidad = m.id_especialidad
JOIN consultorios co ON co.id_consultorio = c.id_consultorio;
```

Vista 2: vista_estadisticas_medico: Agrega el total de citas por médico y las desglosa por estado (atendidas, canceladas, inasistencias) usando SUM con CASE WHEN. Es la fuente principal del dashboard de desempeño en Power BI.

```
CREATE OR REPLACE VIEW vista_estadisticas_medico AS
SELECT
    m.id_medico,
    CONCAT(m.nombre, ' ', m.apellido) AS medico,
    e.nombre AS especialidad,
    COUNT(c.id_cita) AS total_citas,
    SUM(CASE WHEN c.estado = 'Atendida' THEN 1 ELSE 0 END) AS citas_atendidas,
    SUM(CASE WHEN c.estado = 'Cancelada' THEN 1 ELSE 0 END) AS citas_canceladas,
    SUM(CASE WHEN c.estado = 'No asistió' THEN 1 ELSE 0 END) AS inasistencias
FROM medicos m
JOIN especialidades e ON e.id_especialidad = m.id_especialidad
LEFT JOIN citas c ON c.id_medico = m.id_medico
GROUP BY m.id_medico, medico, especialidad;
```

4.1.3 Procedimientos almacenados

SP 1: sp_agendar_cita: Valida, antes de insertar, que el médico no tenga ya una cita activa en esa fecha y hora. Recibe un parámetro de salida (p_resultado) con el mensaje de éxito o error, lo que permite que la API responda con el código HTTP adecuado (201 si se creó, 409 si hubo choque).

```
CREATE PROCEDURE sp_agendar_cita (  
    IN p_id_paciente INT,  
    IN p_id_medico INT,  
    IN p_id_consultorio INT,  
    IN p_fecha DATE,  
    IN p_hora TIME,  
    IN p_motivo VARCHAR(255),  
    OUT p_resultado VARCHAR(100)  
)  
BEGIN  
    DECLARE v_existe INT DEFAULT 0;  
  
    SELECT COUNT(*) INTO v_existe  
    FROM citas  
    WHERE id_medico = p_id_medico  
        AND fecha = p_fecha  
        AND hora = p_hora  
        AND estado NOT IN ('Cancelada');  
  
    IF v_existe > 0 THEN  
        SET p_resultado = 'ERROR: El médico ya tiene una cita en ese horario';  
    ELSE  
        INSERT INTO citas (id_paciente, id_medico, id_consultorio, fecha, hora, motivo)  
        VALUES (p_id_paciente, p_id_medico, p_id_consultorio, p_fecha, p_hora, p_motivo);  
        SET p_resultado = CONCAT('OK: Cita creada con id ', LAST_INSERT_ID());  
    END IF;  
END //
```

SP 2: sp_historial_paciente: Devuelve, para un paciente dado, todas sus citas ordenadas de la más reciente a la más antigua, junto con el diagnóstico asociado si existe (LEFT JOIN, ya que una cita programada aún no tiene diagnóstico).

```
CREATE PROCEDURE sp_historial_paciente (  
    IN p_id_paciente INT  
)  
BEGIN  
    SELECT  
        c.id_cita,  
        c.fecha,  
        c.hora,  
        c.estado,  
        CONCAT(m.nombre, ' ', m.apellido) AS medico,  
        e.nombre AS especialidad,  
        d.descripcion AS diagnostico,  
        d.tratamiento  
    FROM citas c  
    JOIN medicos m ON m.id_medico = c.id_medico  
    JOIN especialidades e ON e.id_especialidad = m.id_especialidad  
    LEFT JOIN diagnosticos d ON d.id_cita = c.id_cita  
    WHERE c.id_paciente = p_id_paciente  
    ORDER BY c.fecha DESC, c.hora DESC;  
END //
```

4.1.4 Datos de prueba

El script incluye datos semilla (seed data): 5 especialidades, 4 consultorios, 5 médicos, 6 horarios, 5 pacientes, 5 citas (en distintos estados) y 1 diagnóstico. Esto permite probar la API, el ETL y el dashboard sin depender de datos reales desde el primer momento.

4.2 API REST — Node.js + Express + Sequelize

La API expone toda la funcionalidad del sistema sobre HTTP, usando Express como framework de enrutamiento y Sequelize como ORM para el CRUD estándar, combinado con consultas crudas (raw queries) para aprovechar las vistas y procedimientos almacenados de la base de datos.

4.2.1 Dependencias principales

Paquete	Versión	Uso
express	^4.19.2	Framework HTTP / enrutamiento
sequelize	^6.37.3	ORM para el CRUD sobre las tablas
mysql2	^3.10.1	Driver de conexión a MariaDB
dotenv	^16.4.5	Carga de variables de entorno (.env)
cors	^2.8.5	Habilita peticiones cross-origin
morgan	^1.10.0	Logging de peticiones HTTP en consola
nodemon	^3.1.4	Reinicio automático en desarrollo (devDependency)

4.2.2 Endpoints disponibles

Formato de respuesta estándar en todos los endpoints: { "status": "success", "data": {...} } en caso de éxito, o { "status": "error", "message": "..." } en caso de error. Códigos HTTP usados: 200 (OK), 201 (creado), 204 (eliminado sin contenido), 400 (solicitud inválida), 404 (no encontrado), 409 (conflicto — choque de horario o referencia inválida) y 500 (error del servidor).

Especialidades — /api/especialidades

Método	Ruta	Descripción
GET	/api/especialidades	Listar todas
GET	/api/especialidades/:id	Obtener una
POST	/api/especialidades	Crear
PUT	/api/especialidades/:id	Actualizar
DELETE	/api/especialidades/:id	Eliminar

Consultorios (/api/consultorios) y Diagnósticos (/api/diagnosticos) siguen exactamente el mismo patrón CRUD.

Médicos — /api/medicos

Método	Ruta	Descripción
GET	/api/medicos	Listar todos
GET	/api/medicos/estadisticas	Consulta cruda sobre vista_estadisticas_medico
GET	/api/medicos/:id	Obtener uno
POST	/api/medicos	Crear
PUT	/api/medicos/:id	Actualizar
DELETE	/api/medicos/:id	Eliminar

Pacientes — /api/pacientes

Método	Ruta	Descripción
GET	/api/pacientes	Listar todos
GET	/api/pacientes/:id	Obtener uno
GET	/api/pacientes/:id/historial	Procedimiento almacenado sp_historial_paciente
POST	/api/pacientes	Crear
PUT	/api/pacientes/:id	Actualizar
DELETE	/api/pacientes/:id	Eliminar

Citas — /api/citas

Método	Ruta	Descripción
GET	/api/citas	Listar todas
GET	/api/citas/completas?fecha=...&estado=...	Consulta cruda sobre vista_citas_completas, con filtros opcionales
POST	/api/citas/agendar	Procedimiento almacenado sp_agendar_cita (valida choque de horario)
GET	/api/citas/:id	Obtener una
POST	/api/citas	Crear (sin validación de choque, vía ORM)
PUT	/api/citas/:id	Actualizar
DELETE	/api/citas/:id	Eliminar

4.2.3 Ejemplo: agendar una cita usando el procedimiento almacenado

```
curl -X POST http://localhost:3000/api/citas/agendar \
-H "Content-Type: application/json" \
-d '{
  "id_paciente": 1,
  "id_medico": 2,
  "id_consultorio": 2,
  "fecha": "2026-08-25",
  "hora": "10:00:00",
  "motivo": "Consulta de control"
}'
```

4.3 ETL — Carga de Pacientes desde Sistema Externo

El script `etl_pacientes.py` simula la llegada de un archivo de pacientes desde un sistema externo (por ejemplo, un formulario web de preregistro, u otra clínica que exporta sus datos). Este tipo de archivos casi siempre llega "sucio": con espacios de más, mayúsculas y minúsculas mezcladas, distintos formatos de fecha, teléfonos mal escritos, correos inválidos y registros duplicados. El proceso sigue el patrón clásico Extract–Transform–Load.

4.3.1 Flujo del proceso

```
data/pacientes_externos.csv --> EXTRACT --> TRANSFORM --> LOAD --> tabla `pacientes`
                                   |
                                   v
                                logs/pacientes_rechazados.csv
                                (registros inválidos, con el motivo)
```

4.3.2 Extract

Lee el CSV de origen tal cual llega (`data/pacientes_externos.csv`), usando `csv.DictReader` para mapear cada fila a un diccionario con las columnas: `nombre_completo`, `fecha_nacimiento`, `genero`, `telefono`, `correo`, `direccion`.

4.3.3 Transform

Por cada fila del CSV, el script:

- Quita espacios extra y normaliza mayúsculas/minúsculas en los nombres.
- Convierte cualquier formato de fecha reconocido (`dd/mm/aaaa`, `aaaa-mm-dd`, `dd-mm-aaaa`, `aaaa/mm/dd`) a un único formato estándar (`aaaa-mm-dd`), y descarta fechas imposibles o futuras.
- Unifica el género a M, F u Otro sin importar cómo vino escrito (masculino, m, Femenino, etc.).
- Normaliza el teléfono al formato `0000-0000`.
- Valida que el correo tenga formato válido mediante una expresión regular.
- Descarta (y registra el motivo de) filas con nombre vacío, correo inválido, género no reconocido, fecha inválida, o correo duplicado dentro del mismo archivo.

```
def parsear_fecha(valor):
    valor = (valor or "").strip()
    for formato in DATE_FORMATS:
        try:
            fecha = datetime.strptime(valor, formato)
            # Descarta fechas imposibles (ej. 31/02) o futuras
            if fecha > datetime.now():
                return None
            return fecha.strftime("%Y-%m-%d")
        except ValueError:
            continue
    return None
```


4.3.4 Load

Inserta los registros ya limpios en la tabla pacientes usando INSERT ... ON DUPLICATE KEY UPDATE sobre el email (que es único en la tabla). Esto hace que el proceso sea idempotente: el script se puede correr varias veces sin duplicar pacientes — si el paciente ya existe, simplemente se actualizan sus datos.

```
sql = """
INSERT INTO pacientes (nombre, apellido, fecha_nacimiento, genero, telefono, email, direccion)
VALUES (%(nombre)s, %(apellido)s, %(fecha_nacimiento)s, %(genero)s, %(telefono)s, %(email)s, %(direccion)s)
ON DUPLICATE KEY UPDATE
    nombre = VALUES(nombre),
    apellido = VALUES(apellido),
    fecha_nacimiento = VALUES(fecha_nacimiento),
    genero = VALUES(genero),
    telefono = VALUES(telefono),
    direccion = VALUES(direccion)
"""
```

4.3.5 Evidencia de ejecución real

El ETL fue ejecutado contra la base de datos real, con un archivo de entrada de 10 filas. El log generado automáticamente (con timestamp, en logs/etl_AAAAMMDD_HHMMSS.log) muestra el resultado exacto:

```
2026-08-19 16:34:09,041 [INFO] ===== INICIO DEL PROCESO ETL =====
2026-08-19 16:34:09,041 [INFO] EXTRACT: leyendo archivo fuente 'C:\Users\ariel\Downloads\proyecto final\proyecto final.csv'
2026-08-19 16:34:09,089 [INFO] EXTRACT: 10 filas leídas del origen
2026-08-19 16:34:09,089 [INFO] TRANSFORM: limpiando y validando registros
2026-08-19 16:34:09,122 [WARNING] Fila 5 rechazada: Email duplicado dentro del mismo archivo -> {'nombre_completo': 'Fatima', 'email': 'fatima@gmail.com'}
2026-08-19 16:34:09,123 [WARNING] Fila 7 rechazada: Email con formato inválido -> {'nombre_completo': 'Fatima', 'email': 'fatima@gmail.com'}
2026-08-19 16:34:09,123 [WARNING] Fila 8 rechazada: Nombre vacío -> {'nombre_completo': '', 'fecha_nacimiento': '2000-01-01'}
2026-08-19 16:34:09,123 [INFO] TRANSFORM: 7 registros válidos, 3 rechazados
2026-08-19 16:34:09,125 [INFO] TRANSFORM: detalle de rechazados guardado en 'C:\Users\ariel\Downloads\proyecto final\proyecto final_rechazados.csv'
2026-08-19 16:34:09,125 [INFO] LOAD: conectando a MariaDB en localhost:3306
2026-08-19 16:34:09,455 [INFO] LOAD: 7 registros insertados/actualizados en 'pacientes'
2026-08-19 16:34:09,525 [INFO] ===== FIN DEL PROCESO ETL =====
```

4.3.6 Automatización

Para producción, el script está pensado para correr sin intervención manual:

- **Windows:** Programador de tareas → tarea básica → acción "Iniciar un programa" → programa python, argumento etl_pacientes.py.
- **Linux/Mac:** entrada de cron, por ejemplo 0 2 * * * para ejecutarlo todos los días a las 2 a. m.

4.4 Visualización — Power BI

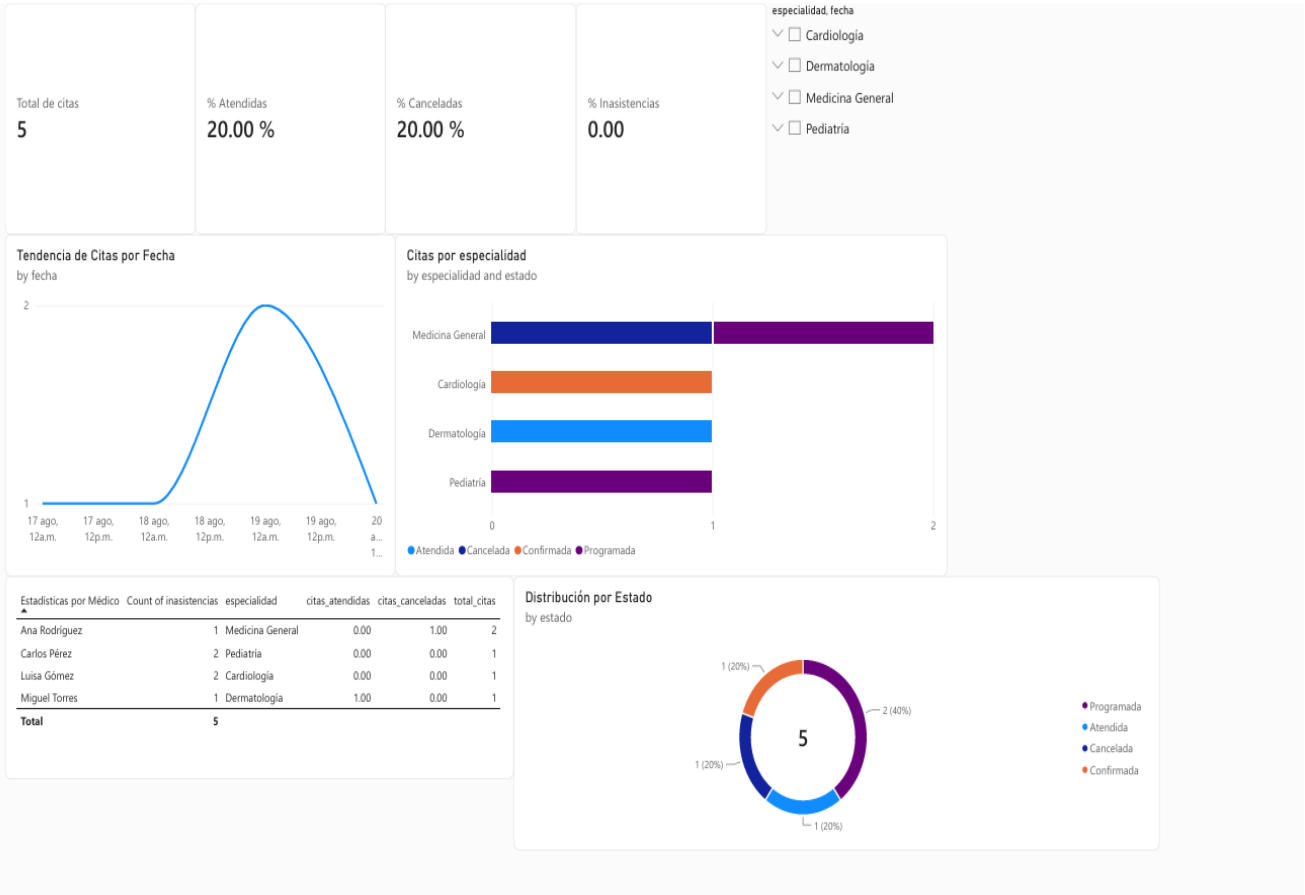
El dashboard se construyó conectando Power BI Desktop directamente a la base de datos MariaDB, usando las vistas `vista_citas_completas` y `vista_estadisticas_medico` como fuente principal de datos — de esa forma el reporte siempre refleja el estado actual de la base de datos sin necesidad de exportar archivos intermedios.

4.4.1 Conexión de Power BI a MariaDB

Pasos generales para reproducir la conexión:

- Instalar el conector ODBC de MariaDB/MySQL (MySQL Connector/ODBC) de 64 bits, requerido por Power BI para hablar con el motor.
- En Power BI Desktop: Obtener datos → Base de datos MySQL (el conector es compatible con MariaDB).
- Especificar el servidor (`localhost:3306`) y la base de datos (`clinica_citas`).
- Autenticar con el usuario y contraseña de MariaDB (sin exponer credenciales reales en la documentación).
- Seleccionar las vistas `vista_citas_completas` y `vista_estadisticas_medico` como tablas de origen, en modo de importación.

4.4.2 Contenido del dashboard



4.5 Pruebas — Funcionales y de Rendimiento

4.5.1 Pruebas funcionales (REST Client / Postman)

Las pruebas funcionales se definieron en el archivo pruebas.http (formato REST Client de VS Code, equivalente a una colección de Postman), cubriendo tanto los casos exitosos como los casos de error esperados de cada módulo de la API.

Módulo	Casos cubiertos
Health check	GET /api/health — verifica que el servidor está activo
Especialidades	Listar todas, crear una nueva
Médicos	Listar todos, estadísticas (consulta cruda), obtener uno
Pacientes	Listar todos, historial (procedimiento almacenado), crear
Citas	Listar todas, detalle completo con filtros, agendar cita válida, agendar cita con choque de horario (caso negativo, debe responder 409), actualizar estado, eliminar
Diagnósticos	Listar todos, crear uno

```
### Agendar cita con choque de horario (debe responder 409)
Send Request
POST {{baseUrl}}/citas/agendar
Content-Type: application/json

{
  "id_paciente": 2,
  "id_medico": 1,
  "id_consultorio": 1,
  "fecha": "2026-08-17",
  "hora": "08:30:00",
  "motivo": "Choque de prueba"
}
```

```

1  HTTP/1.1 409 Conflict
2  X-Powered-By: Express
3  Access-Control-Allow-Origin: *
4  Content-Type: application/json; charset=utf-8
5  Content-Length: 81
6  ETag: W/"51-51CSMQ1nFmHLfo0XlFJeb4zuhRc"
7  Date: Thu, 20 Aug 2026 20:46:53 GMT
8  Connection: close
9
10 {
11   "status": "error",
12   "message": "ERROR: El médico ya tiene una cita en
    ese horario"
13 }

```

4.5.2 Pruebas de rendimiento (Apache JMeter)

Se configuró un plan de pruebas en JMeter contra la API corriendo localmente (localhost:3000), cubriendo 6 endpoints representativos del sistema:

- GET /api/health
- GET /api/especialidades
- GET /api/medicos
- GET /api/medicos/estadisticas
- GET /api/pacientes
- GET /api/citas/completas

Configuración del Thread Group

Parámetro	Valor
Number of Threads (usuarios simulados)	20
Ramp-up period	10 segundos
Loop Count	5

Se agregaron dos listeners para recolectar resultados: Summary Report y Aggregate Report (este último con los percentiles de tiempo de respuesta, requeridos para evaluar el rendimiento bajo carga).

Resultados obtenidos

Métrica	Resultado
Peticiones totales	1,800
Tiempo promedio	3 ms
Mediana	3 ms
Percentil 90%	5 ms
Percentil 95%	6 ms
Percentil 99%	19 ms
Máximo	227 ms
Error %	0.00%
Throughput	7.3 peticiones/segundo

Clinica Citas - Plan de Pruebas.jmx (C:\Users\ariel\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\Clinica Citas - Plan de Pruebas.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

Clinica Citas - Plan de Pruebas

Thread Group

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

Summary Report

Aggregate Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: resultados/summary.csv

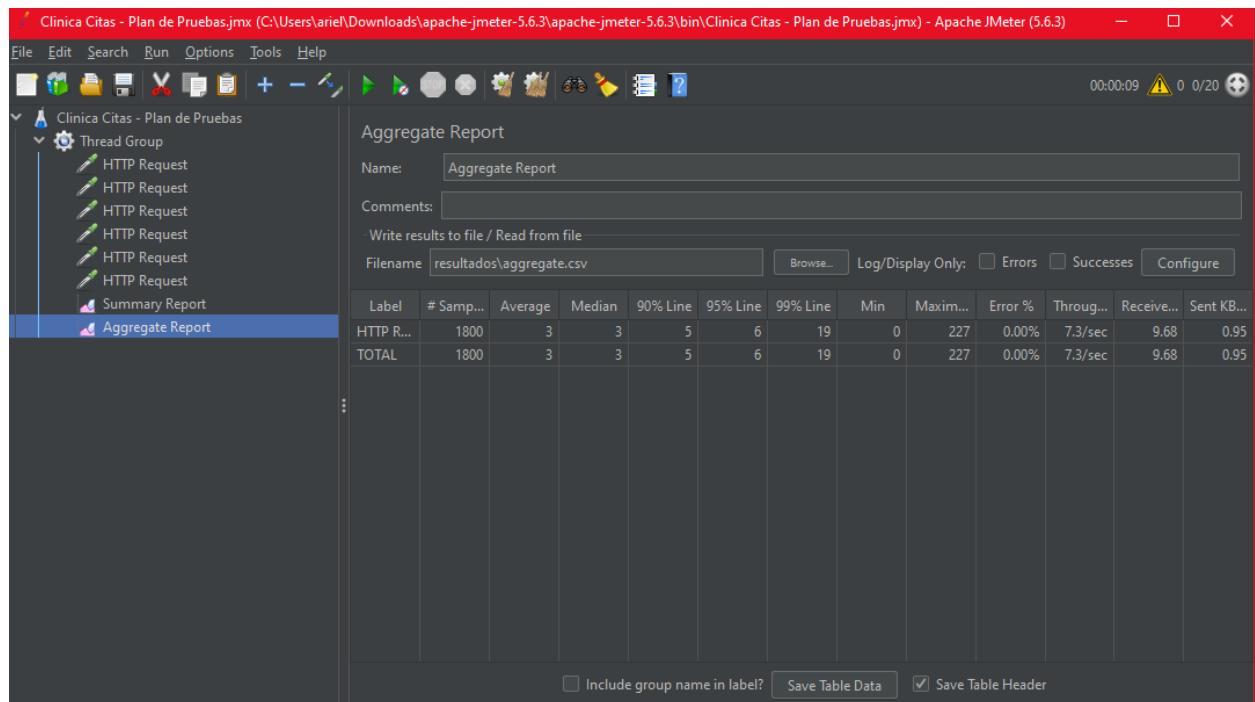
Browse...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughp...	Received ...	Sent KB/s...	Avg. Bytes
HTTP Req...	1800	3	0	227	6.71	0.00%	7.3/sec	9.68	0.95	1349.5
TOTAL	1800	3	0	227	6.71	0.00%	7.3/sec	9.68	0.95	1349.5

☐ Include group name in label? ☒ Save Table Header



La evidencia de esta prueba (capturas y archivos CSV exportados por JMeter: resultados/summary.csv y resultados/aggregate.csv) se conserva en la carpeta proyecto_clinica/jmeter/resultados del repositorio del proyecto.

5. Manual de Instalación y Ejecución — Paso a Paso

Este manual describe el orden real en que se instaló y probó el proyecto de principio a fin: primero la base de datos, luego la API con sus pruebas, después el ETL, seguido del dashboard de Power BI, y finalmente las pruebas de carga con JMeter.

Paso 1 — Instalar y cargar la base de datos MariaDB

Instalar MariaDB (en este proyecto se usó la versión 12.3) y, una vez instalado, importar el script del esquema. En Windows, con PowerShell, el operador < no funciona igual que en una terminal Unix, así que la forma correcta de importar el script es:

```
# Ubicar el binario de mysql.exe dentro de la instalación de MariaDB
Get-ChildItem "C:\Program Files\" -Filter "MariaDB*" -Directory

# Importar el script (PowerShell no soporta '<' como en bash/cmd)
Get-Content database\01_schema.sql | & "C:\Program Files\MariaDB 12.3\bin\mysql.exe" -u root -p
```

Verificar que las tablas y vistas se crearon correctamente:

```
& "C:\Program Files\MariaDB 12.3\bin\mysql.exe" -u root -p -e "USE clinica_citas; SHOW
TABLES;"

+-----+
| Tables_in_clinica_citas |
+-----+
| citas                    |
| consultorios             |
| diagnosticos             |
| especialidades          |
| horarios_medicos        |
| medicos                 |
| pacientes               |
| vista_citas_completas    |
| vista_estadisticas_medico |
+-----+
```

Paso 2 — Levantar la API REST

```
cd api
copy .env.example .env      # y editar con las credenciales reales de MariaDB
npm install
npm run dev                 # usa nodemon; también se puede usar: npm start
```

Al iniciar correctamente, la consola muestra:


```
[nodemon] watching path(s): *.*  
API corriendo en http://localhost:3000  
GET /api/health 200 36.986 ms - 69
```

Paso 3 — Probar la API

Con la API corriendo, se ejecutaron las pruebas funcionales del archivo pruebas.http directamente contra los endpoints. La consola de la API confirma cada petición recibida y su código de respuesta:

```
GET /api/health 200 0.762 ms - 69  
GET /api/especialidades 200 17.232 ms - 489  
POST /api/especialidades 201 100.698 ms - 105  
GET /api/medicos 200 4.069 ms - 920  
GET /api/medicos/estadisticas 200 9.281 ms - 805  
GET /api/citas/completas?estado=Confirmada 200 10.964 ms - 312
```

Paso 4 — Configurar y ejecutar el ETL

```
cd etl  
copy .env.example .env      # editar con las credenciales de MariaDB  
python -m pip install -r requirements.txt  
python etl_pacientes.py
```

Salida esperada (resumen del log generado):

```
EXTRACT: 10 filas leídas del origen  
TRANSFORM: 7 registros válidos, 3 rechazados  
LOAD: 7 registros insertados/actualizados en 'pacientes'
```

Paso 5 — Construir el dashboard en Power BI

- Abrir Power BI Desktop y conectar contra el servidor MariaDB (Obtener datos → Base de datos MySQL), apuntando a la base clinica_citas.
- Importar las vistas vista_citas_completas y vista_estadisticas_medico.
- Construir los visuales (gráficos de barras, tarjetas KPI, tablas) sobre esos datos.
- Publicar o guardar el archivo .pbix como entregable del proyecto.

Paso 6 — Pruebas de carga con Apache JMeter

Con la API todavía corriendo en localhost:3000, se abrió Apache JMeter (versión 5.6.3, sobre Java 21) y se configuró el plan de pruebas:

- Crear un Thread Group y, dentro de él, 6 HTTP Request (uno por cada endpoint a probar), usando localhost / 3000 / GET y el path correspondiente.
- Configurar el Thread Group: Number of Threads = 20, Ramp-up period = 10, Loop Count = 5.

- Agregar los listeners Summary Report y Aggregate Report para capturar los resultados y percentiles.
- Ejecutar la prueba y guardar los resultados en archivo (Filename: resultados\summary.csv y resultados\aggregate.csv en cada listener).

Verificación de que Java estaba disponible (requisito de JMeter):

```
java -version
java version "21.0.2" 2024-01-16 LTS
```

Resumen del orden de ejecución

#	Paso	Herramienta
1	Instalación y carga del esquema de base de datos	MariaDB
2	Instalación y arranque de la API REST	Node.js / Express
3	Pruebas funcionales de la API	REST Client / Postman
4	Ejecución del proceso ETL	Python
5	Construcción del dashboard	Power BI
6	Pruebas de carga y rendimiento	Apache JMeter

6. Lecciones Aprendidas

Durante el desarrollo del proyecto surgieron varios obstáculos, principalmente relacionados con el entorno de ejecución en Windows.

6.1 PowerShell no interpreta la redirección '<' como bash

Al intentar importar el script SQL con `mysql -u root -p < database\01_schema.sql`, PowerShell devolvió el error "El operador '<' está reservado para uso futuro". Esto ocurre porque PowerShell no es una terminal compatible con bash/cmd en el manejo de redirecciones de entrada.

Solución aplicada: usar el cmdlet `Get-Content` para leer el archivo y canalizarlo (pipe) hacia `mysql.exe`: `Get-Content database\01_schema.sql | mysql -u root -p`. Lección: al documentar comandos para un entorno Windows/PowerShell, no asumir que la sintaxis de bash funciona igual; conviene probar y, si aplica, dar la alternativa nativa de PowerShell.

6.2 El ejecutable de MariaDB no estaba en el PATH del sistema

Después de resolver el problema de la redirección, el comando seguía fallando porque `mysql` no se reconocía como comando ("El término 'mysql' no se reconoce..."). La instalación de MariaDB en Windows no agrega automáticamente su carpeta bin al PATH del sistema.

Solución aplicada: ubicar la ruta exacta del ejecutable con `Get-ChildItem "C:\Program Files\" -Filter "MariaDB*" -Directory`, y luego invocarlo con su ruta completa entre comillas usando el operador de invocación `&`: `& "C:\Program Files\MariaDB 12.3\bin\mysql.exe" -u root -p`. Lección: cuando un comando de línea de base de datos "no se reconoce", el primer diagnóstico debe ser verificar el PATH antes de asumir que el software no está instalado.

6.3 pip tampoco estaba disponible directamente en PATH

Al intentar instalar las dependencias de Python con `pip install -r requirements.txt`, PowerShell tampoco reconoció el comando `pip`, aunque Python sí estaba instalado y funcionando (`python --version` respondía correctamente).

Solución aplicada: invocar `pip` como módulo de Python en lugar de como comando independiente: `python -m pip install -r requirements.txt`. Esta forma es más robusta porque usa siempre el `pip` asociado al intérprete de Python activo, evitando ambigüedad cuando hay varias instalaciones de Python en el sistema.

6.4 Rutas de proyecto de ejemplo no coinciden con la estructura real

En un punto de la instalación se intentó navegar a una ruta de ejemplo (`C:\Users\TU_USUARIO\Desktop\proyecto_clinica`) que en realidad era un marcador de plantilla, no la ruta real del proyecto, lo que generó un error de "ruta no encontrada".

Lección: al seguir (o al escribir) guías de instalación, verificar siempre la estructura de carpetas real con `dir / ls` antes de ejecutar comandos de navegación, en lugar de asumir que una ruta de ejemplo es literal.

6.5 La validación de choque de horario debía vivir en la base de datos, no solo en la API

Aunque `sp_agendar_cita` valida el choque de horario antes de insertar, se reforzó además con la restricción `UNIQUE uq_medico_fecha_hora` directamente en la tabla `citas`.

Lección: las reglas de negocio críticas (como evitar que un médico tenga dos citas al mismo tiempo) no deben depender únicamente de la lógica de la aplicación; conviene reforzarlas también a nivel de esquema de base de datos, para que la integridad se mantenga incluso si en el futuro otra aplicación o proceso escribe directamente en la tabla sin pasar por el procedimiento almacenado.

6.6 Un ETL de calidad necesita reportar, no solo descartar, los datos inválidos

Durante la prueba real del ETL, de 10 filas de entrada se rechazaron 3 (un email duplicado, un email sin formato válido y un nombre vacío). En vez de simplemente ignorarlas, el script las guarda con su motivo exacto en `logs/pacientes_rechazados.csv`.

Lección: en un proceso de carga de datos externos, es tan importante lo que se rechaza como lo que se carga. Guardar el motivo específico de cada rechazo permite corregir el problema en el origen, en vez de simplemente perder esos registros de forma silenciosa.

7. Glosario de Términos

Término	Definición
3FN (Tercera Forma Normal)	Nivel de normalización de una base de datos relacional en el que se eliminan las dependencias transitivas: todo atributo no clave debe depender únicamente de la llave primaria, y no de otro atributo no clave.
API REST	Interfaz de programación de aplicaciones que expone funcionalidades a través de peticiones HTTP (GET, POST, PUT, DELETE), siguiendo los principios de la arquitectura REST (Representational State Transfer).
Llave primaria (PK)	Columna (o conjunto de columnas) que identifica de forma única cada fila de una tabla.
Llave foránea (FK)	Columna que referencia la llave primaria de otra tabla, estableciendo una relación entre ambas.
CRUD	Acrónimo de Create, Read, Update, Delete: las cuatro operaciones básicas de manipulación de datos persistentes.
ORM (Object-Relational Mapping)	Técnica y biblioteca (en este proyecto, Sequelize) que permite manipular una base de datos relacional usando objetos y clases del lenguaje de programación, en vez de escribir SQL manualmente.
Consulta cruda (Raw Query)	Consulta SQL escrita y ejecutada directamente, sin pasar por el mapeo del ORM; se usa cuando se necesita aprovechar una vista, un procedimiento almacenado o una consulta muy específica.
Vista (View)	Consulta SQL almacenada que se comporta como una tabla virtual; no almacena datos propios, sino que los calcula a partir de otras tablas cada vez que se consulta.
Procedimiento almacenado (Stored Procedure)	Bloque de código SQL con lógica (variables, condicionales, parámetros de entrada/salida) que se guarda en la base de datos y se puede invocar por su nombre, encapsulando reglas de negocio a nivel del motor de base de datos.
ETL (Extract, Transform, Load)	Proceso de tres etapas para mover datos entre sistemas: extraerlos de un origen, transformarlos/limpiarlos, y cargarlos en un destino.
Idempotente	Se dice de una operación que produce el mismo resultado final sin importar cuántas veces se ejecute; en este proyecto, el ETL es idempotente porque usa ON DUPLICATE KEY UPDATE en vez de duplicar registros en cada corrida.
Motor de almacenamiento InnoDB	Motor de almacenamiento de MySQL/MariaDB que soporta transacciones, llaves foráneas e integridad referencial; usado en todas las tablas de este proyecto.
ENUM	Tipo de dato que restringe una columna a un conjunto fijo y predefinido de valores posibles (por ejemplo, el estado de una cita).
ON DELETE RESTRICT	Regla de integridad referencial que impide eliminar una fila si todavía existen otras filas en otra tabla que la referencian mediante una llave foránea.
ON DELETE CASCADE	Regla de integridad referencial que, al eliminar una fila, elimina automáticamente todas las filas relacionadas en otras tablas que dependen de ella.
Dashboard	Panel visual (en este proyecto, construido en Power BI) que resume datos e indicadores clave mediante gráficos, tablas y tarjetas, para facilitar la toma de decisiones.

Término	Definición
Thread Group (JMeter)	Elemento de configuración en Apache JMeter que define cuántos usuarios virtuales (hilos) van a ejecutar un conjunto de peticiones, con qué tiempo de arranque (ramp-up) y cuántas veces (loop count).
Percentil (p90, p95, p99)	Métrica de rendimiento que indica el tiempo de respuesta bajo el cual cae un determinado porcentaje de las peticiones; por ejemplo, un p99 de 19 ms significa que el 99% de las peticiones respondió en 19 ms o menos.
Throughput	Cantidad de peticiones procesadas por unidad de tiempo (en este proyecto, medido en peticiones por segundo) durante una prueba de carga.
Variables de entorno (.env)	Archivo de configuración que almacena valores sensibles o dependientes del entorno (como credenciales de base de datos), separados del código fuente, y que no debe subirse a un repositorio público.
Colección Postman	Conjunto organizado de peticiones HTTP guardadas (en este proyecto, representado también en formato REST Client mediante el archivo pruebas.http) que permite probar sistemáticamente los endpoints de una API.

<https://github.com/SHOT10/proyecto-final.git>